

Confidence Is Theater: What I Learned Building Scribe

I'm building Scribe, a local pipeline that turns handwritten clinical forms into structured data.

The obvious problem is reading the handwriting.

The harder problem turned out to be this:

How do you know when the model is wrong?

My first idea was simple: ask the model how confident it was.

That failed almost immediately.

Median confidence when the answer was correct: **0.95**. Median confidence when the answer was wrong: **0.95**.

So confidence wasn't giving me a useful signal.

That sent the project in a different direction: instead of asking a model whether I should trust it, I started making two reads disagree with each other.

This is the engineering log of what happened next.

What Scribe does

Scribe takes scanned or photographed handwritten clinical forms and converts them into structured records.

The pipeline runs entirely on one machine. That matters for environments where connectivity is unreliable or the data should not leave the device.

Each extracted value keeps a link back to where it came from on the page. If the system is not comfortable with a value, it sends that field for human review instead of quietly writing it into the record.

The basic flow is:

page → find fields → read → validate → second read → human review if needed → structured record

There are more technical names for those stages, but that is really what the system is doing.

A quick note about the numbers

I tested nine pipeline configurations on the same 23 handwritten pages and the same 221 hand-curated field values.

The dataset used for this development bench is [public](#), which means some models may have seen similar material during training.

So I'm treating these numbers as engineering comparisons between configurations, not as claims about the absolute performance of the models.

Production sign-off will use a private held-out set created after model training cutoffs.

1. Whole page or crop?

One of the first questions was whether the model should read the whole page or just the area containing the field.

Both approaches had problems.

Reading the whole page preserved context. Names, IDs and header-labeled fields were easier to interpret.

But the extra context also created hallucinations. A model looking for a document date could "borrow" another date from a nearby table.

Reading only a crop fixed some of that. For example, document-date hallucination dropped from 67% to 17%.

But crops sometimes removed too much context. Hospital-number accuracy, for example, dropped from 87% to 65%.

So I stopped trying to choose one.

Scribe now routes by field type:

- tables, dates, enums and checkbox groups → crop
- scalars and free text → whole page

That hybrid approach beat both pure modes on the combined quality measures and became the default.

2. Confidence didn't work

The next problem was deciding which extracted values could safely skip human review.

Originally, I planned to use the model's own confidence score.

That sounded reasonable.

It wasn't.

The median confidence was 0.95 for correct answers and 0.95 for wrong ones. Even badly misread handwritten names could come back with very high confidence.

There was no useful threshold to tune.

So I changed the question.

Instead of asking:

"How sure are you?"

I asked:

"Does another read agree?"

3. A second read helped much more than confidence

The first verification approach simply re-read every field that would otherwise have been accepted automatically.

The second read used the complementary source:

- first read from crop → verify from whole page
- first read from whole page → verify from crop

If the two reads disagreed, the field went to human review.

That one change moved the accuracy of automatically accepted fields from 79% to 93%, while silent errors dropped from 21% to 7%.

The underlying model had not suddenly become a better reader.

The pipeline had become better at recognizing when it should not trust the answer.

That became the main design principle of Scribe.

4. Then I tried a different model for the second read

There was still an obvious problem.

If the same model has the same bias twice, two reads can agree on the same wrong answer.

So I switched the verifier to a smaller, different model.

That moved auto-accepted accuracy again, from 93% to 95%.

More importantly, confidently misread patient names were now caught before they passed through automatically.

The smaller model was not necessarily a better reader.

Sometimes it was useful precisely because it was wrong in a different way.

That was enough to surface disagreements.

5. Some mistakes were shared by nearly everyone

The most stubborn example involved checkboxes.

One synthetic form contained a lab-request section where nothing was checked. The correct answer was null.

Specimen type:	☐ Blood	☐ Urine	☐ Stool	☐ CSF
----------------	--------------------------------	--------------------------------	--------------------------------	------------------------------

Several model families still confidently chose an option.

Qwen selected "blood". GLM showed the same kind of behavior. Gemma interpreted an empty box as a tick.

Cross-model verification did not help much because both readers could share the same bias.

That changed the engineering plan. Checkboxes stopped being treated as “just another prompt.” They became their own detection problem.

Later, Muse Glimmer produced an interesting counterexample: it hallucinated nothing in that run, but performed much worse overall. That makes it potentially useful as a specialist rather than as the main reader.

That was a useful reminder:

A second model only helps when the models fail differently.

6. Changing the verifier eventually stopped helping

After the initial gains, I tested several different verifier families.

GLM-4.5V. Gemma 3. Gemma 4.

They all ended up within roughly two fields of the small Qwen verifier on this 221-value set.

GLM was actually worse operationally because it sent far more fields to review without improving the overall result much.

That told me something useful. The next improvement probably wasn't hiding in yet another verifier.

The remaining gains were more likely to come from:

- a better primary model
- explicit checkbox detection
- resolving a few annotation conventions

So I stopped verifier shopping.

7. The metric I care about changed too

At first I tracked hallucination as one number: did the model invent a value where the correct answer was empty?

Later I split that into two questions:

1. Did the model invent something?
2. Did that invention escape all the safety checks?

The second one matters much more to the final system.

For the routed pipeline without verification, 12% of null fields were invented and 12% escaped.

With cross-model verification, the model still invented values, but only 2% escaped.

That distinction changed how I think about evaluation.

If you only measure raw hallucination, you are grading the model.

If you measure what survives validation, verification and review, you are grading the system.

And the system is what actually ships.

Where the bench ended up

Across the nine configurations, raw exact accuracy only moved from roughly 75% to 81%.

But the accuracy of fields allowed to bypass human review reached 96% in the current default configuration.

That is the result I find most interesting.

The project did not succeed because I found a model that suddenly became perfect at handwriting.

It improved because the surrounding pipeline got better at recognizing when the model should not be trusted.

What still needs to be tested

There are still three important open questions.

Contamination. The development corpus is public, so the next evaluation needs data the models could not have seen.

Production targets. The target is at least 98% accuracy on automatically accepted fields and no more than 2% escaped hallucination, measured only on the private held-out set.

Generalization. This one already has a first answer: in [part 2 of this series](#), I ran the two best configurations against 100 independently labeled handwritten prescription pages that nobody tuned for. Reading collapsed. Something more interesting held.

So the next useful number should come from genuinely unseen data, not from squeezing another decimal point out of this development set.

What I'm taking away from it

A few things changed my thinking while building Scribe:

- model confidence was not useful for deciding whether an answer was safe
- disagreement between independent reads was much more useful
- crops and whole-page context solve different problems
- changing models does not fix every failure mode
- some problems need deterministic engineering rather than another prompt
- the most important hallucinations are the ones that escape the system

Scribe now runs locally, and new form types are defined through schema YAML rather than custom extraction code. The evaluation setup also keeps model revisions, provenance and review decisions tied to each result.

I'm still testing it.

But the central lesson so far is simple:

The goal is not to build a model that never makes mistakes. It is to build a system that knows when a model's answer needs another look.

► **The full scoreboard, for engineers (click to expand)**

This is part 1 of a series. Part 2: [*One imperfect page per hundred*](#) — the external evaluation on data nobody tuned against.

Code and methodology: github.com/kod201/scribe · Benchmark gold set: [sohakanu/scribe-flow-gold](#)

Every name on every sample page shown or discussed is fictional. All extraction ran locally on one machine; no page left it.